

AFRILANG Stdlib

std/c/cryptparity

Bibliothèque AFRILANG `std/c/cryptparity` (niveau complex, catégorie complex). Elle expose 8 fonction(s) :
pariLen, pari

```
`import "std/c/cryptparity" . `use cryptparity`
```

- `pariLen(s text) ? number`
- `pariUpper(s text) ? text`
- `pariLower(s text) ? text`
- `pariTrim(s text) ? text`
- `pariSplit(s text, delim text) ? list text`
- `pariJoin(parts list text, delim text) ? text`
- `pariReplace(s text, from text, to text) ? text`
- `hash32(s text) ? number`

Category: complex | Tier: complex | Functions: 8

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/cryptparity` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : `pariLen`, `pari`

```
`import "std/c/cryptparity" . `use cryptparity`  
- `pariLen(s text) ? number`  
- `pariUpper(s text) ? text`  
- `pariLower(s text) ? text`  
- `pariTrim(s text) ? text`  
- `pariSplit(s text, delim text) ? list text`  
- `pariJoin(parts list text, delim text) ? text`  
- `pariReplace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptparity"  
use cryptparity
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? pariLen(s text) ? number  
? pariUpper(s text) ? text  
? pariLower(s text) ? text  
? pariTrim(s text) ? text  
? pariSplit(s text, delim text) ? list  
? pariJoin(parts list text, delim text) ? text  
? pariReplace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Exemples d'utilisation

```
import "std/c/cryptparity"  
use cryptparity  
  
create result = pariLen(/* args */)   
say result  
  
create result = pariUpper(/* args */)   
say result  
  
create result = pariLower(/* args */)   
say result  
  
create result = pariTrim(/* args */)   
say result  
  
create result = pariSplit(/* args */)   
say result  
  
create result = pariJoin(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez ``import "std/c/cryptparity" `` en tête de fichier.
- ? Lancez ``afirang check`` avant de livrer.
- ? Combinez avec les modules `stdlib` de la même catégorie.
- ? Voir `/stdlib/` pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module cryptparity
function pariLen(s text) returns number
end
function pariUpper(s text) returns text
end
function pariLower(s text) returns text
end
function pariTrim(s text) returns text
end
function pariSplit(s text, delim text) returns list text
end
function pariJoin(parts list text, delim text) returns text
end
function pariReplace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library ``std/c/cryptparity`` (tier complex, category complex). Exports 8 function(s): `pariLen`, `pariUpper`, `pariLo`

```
`import "std/c/cryptparity"` . `use cryptparity`  
- `pariLen(s text) ? number`  
- `pariUpper(s text) ? text`  
- `pariLower(s text) ? text`  
- `pariTrim(s text) ? text`  
- `pariSplit(s text, delim text) ? list text`  
- `pariJoin(parts list text, delim text) ? text`  
- `pariReplace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/cryptparity"  
use cryptparity
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? pariLen(s text) ? number  
? pariUpper(s text) ? text  
? pariLower(s text) ? text  
? pariTrim(s text) ? text  
? pariSplit(s text, delim text) ? list  
? pariJoin(parts list text, delim text) ? text  
? pariReplace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Usage examples

```
import "std/c/cryptparity"  
use cryptparity  
  
create result = pariLen(/* args */)   
say result  
  
create result = pariUpper(/* args */)   
say result  
  
create result = pariLower(/* args */)   
say result  
  
create result = pariTrim(/* args */)   
say result  
  
create result = pariSplit(/* args */)   
say result  
  
create result = pariJoin(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/cryptparity"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module cryptparity
function pariLen(s text) returns number
end
function pariUpper(s text) returns text
end
function pariLower(s text) returns text
end
function pariTrim(s text) returns text
end
function pariSplit(s text, delim text) returns list text
end
function pariJoin(parts list text, delim text) returns text
end
function pariReplace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```